

IDEC ZNX Checksum Process

Sensored

October 4, 2026

Abstract

Package integrity notes for HG2J / Linux-series HMI updates
Validated on Automation Organizer / WindOI-NV4 5.1 output and live
HMI installations

1 Purpose and status

This document records the checksum mechanisms currently confirmed for IDEC ZNX update packages used by WindOI-NV4 ZNX and znv files. The primary goal is to support reliable extraction, modification, and repacking of ZNX files without depending on Original Source derived manifest files. The findings below are based on Automation Organizer 5.1 code interactions, binary inspection of HMI runtime components, controlled file modifications, and a successful extract/repack/download round-trip on an HG2J HMI.

Current confidence: High for the top-level ZNX member checksum algorithm and record fields described here. The complete semantics of every byte in the 0x1C-byte ZNX header are not yet fully defined.

2 ZNX container overview

A ZNX package contains a short header followed by a directory of member records and then the member payloads. In the tested HG2J packages, the top-level members are:

Member	Purpose	Integrity field
os_update.tar.xz	Linux/HMI OS and runtime update payload	IDEC XOR32
project.znv	HMI project archive and runtime/project files	IDEC XOR32

The member record format confirmed from WindOI-NV4 5.1 writer code is:

```
u32 file_type
u32 relative_offset
```

```

u32 file_size
u32 checksum
u32 padded_filename_length
char filename[padded_filename_length]

```

Offsets are relative to the ZNX body base at file offset 0x1C. Filenames are stored in 4-byte-aligned slots. The payloads follow the directory records.

3 IDEC ZNX checksum algorithm

The four-byte checksum stored in each ZNX member record is not CRC32, Adler-32, MD5, SHA-256, or a compressed-stream checksum. WindOI-NV4 computes it by XORing every complete little-endian 32-bit word in the member payload.

```

checksum = 0
for each complete 4-byte word in payload:
    checksum ^= little_endian_u32(word)
return checksum

```

If a file length is not divisible by four, the final 1-3 bytes are ignored by this algorithm. This behavior follows directly from the loop bound used by both implementations.

4 Extracted writer evidence

In WindOI-NV4.exe, WNVManager.WriteDetailFileInfoForZNX() writes the source-file length, calls WNVManager.CalculateChecksum(sourceFile), and then writes that UInt32 result into the ZNX member record. CalculateChecksum() opens the file and delegates to CalculateChecksumBinary(). CalculateChecksumBinary() reads UInt32 values and XORs them together.

A second implementation in WindLDR/Communications.dll, PDCommunication.CalcSum4Bytes(byte[]), independently performs the same operation using BitConverter.ToUInt32(data, index * 4) followed by XOR. This independent implementation was useful confirmation that the field is intentional and not a compiler artifact.

5 Python reference implementation

```

import struct

def idec_checksum(data):
    value = 0
    for offset in range(0, len(data) - 3, 4):

```

```

        value ^= struct.unpack_from('<I', data, offset)[0]
    return value

```

For a file on disk, read the exact stored member bytes and pass them to this function. Do not decompress or reinterpret the payload before calculating the ZNX member checksum.

6 Confirmed checksum examples

Payload	Confirmed checksum	Result
os_update.tar.xz	0xC02C91A6	Matched stored ZNX field
project.znv	0x5536C42E	Matched stored ZNX field
HG2F.orig	0xE6EA3BA9	Matched stored ZNV field

These values were reproduced from the extracted payload bytes using the reference Python algorithm. The calculated values exactly matched the values already present in the corresponding package records.

7 Nested integrity: ZNX containing ZNV

The integrity chain is layered. A ZNX record checks the complete stored project.znv file. The ZNV file in turn contains its own per-file records, including a checksum for HG2F.BIN and other project/runtime components.

```

ZNX
os_update.tar.xz  -> XOR32 of stored archive bytes
project.znv       -> XOR32 of complete stored ZNV bytes
                  |
                  +-- ZNV member records
                  HG2F.BIN -> XOR32 of stored member bytes

```

WindOI-NV4's ZNV writer uses the same checksum operation. For an unencrypted member it calculates the checksum from the source file. When the member is encrypted before storage, the checksum is calculated over the encrypted byte array. Therefore, any manual ZNV edit must repair the checksum of the modified inner member before the outer project.znv checksum is recalculated.

8 Outer MD5 sidecar

The HMI download process also uses a separate MD5 sidecar for the complete transferred os_update.znx file. The downloader calculates the MD5 of the complete finished ZNX and uploads a corresponding os_update.znx.md5 file. HMI-side code in idec_boot.BIN verifies this outer MD5 before installation.

```
modified inner file
-> repair inner ZNV XOR32
-> rebuild project.znv
-> repair project.znv XOR32 in ZNX
-> rebuild ZNX
-> calculate whole-file MD5
-> upload ZNX + .md5 sidecar
```

The MD5 sidecar does not replace the internal XOR32 checksums. A package can have a correct whole-file MD5 and still fail installation if one of the internal record checksums is stale.

9 Controlled failure that exposed the nested checks

A one-byte edit was made inside HG2F.BIN, changing the ASCII text 'usb_wlan1' to 'usb_wlon1'. The edit preserved file length. The downloader generated a fresh whole-ZNX MD5 and the transfer completed, but the HMI rejected the installation at LC Kernel. At that time the HG2F.BIN ZNV checksum and the outer project.znv ZNX checksum had not been updated.

This established that a fresh outer MD5 is insufficient. Subsequent compilation identified the exact XOR32 algorithm used to populate both checksum fields.

10 Successful round-trip validation

A known-good rgb_51.ZNX file was extracted into its two raw members and repacked without changing either payload. The repacker recalculated the member checksums and reconstructed the container.

```
Original SHA256:
a78431b9933c5082ccc0c2b8422faadecf902523eb281981abe84a978d3007d2
```

```
Repacked SHA256:
a78431b9933c5082ccc0c2b8422faadecf902523eb281981abe84a978d3007d2
```

Result: byte-for-byte identical

The repacked ZNX was then transferred with hmi_znx_download.py. The HMI accepted the package and returned ACK for LC Kernel, LD, and AH. This is the first confirmed successful ZNX extract/repack/install cycle produced by the process tooling.

11 Basic repacking order

1. extract a znx file.

2. insert a new `project.znv` or modified linux source files
3. repack your project.
4. download

12 Recommended repacking order

1. Start with a known-valid `project.znv` and `os_update.tar.xz`.
 2. If modifying a file inside `project.znv`, recalculate that ZNV member's XOR32 checksum.
 3. Rebuild or patch `project.znv` so all internal offsets, sizes, and checksums are valid.
 4. Calculate XOR32 over the complete final `project.znv`.
 5. Calculate XOR32 over the exact stored `os_update.tar.xz` bytes.
 6. Build the ZNX member directory with correct offsets, sizes, name-slot lengths, and checksums.
 7. Write the two payloads in the expected order.
 8. Update the ZNX body-size/header fields that are derived from final file size.
 9. Calculate the MD5 of the completed ZNX and create the `.md5` sidecar used for transfer.
 10. Only then perform the maintenance-protocol download/install sequence.
- scripts in this project can perform this process.

13 Current unknowns and cautions

The checksum mechanism itself is now well established, but not every ZNX header field has been assigned a semantic meaning. The current round-trip repacker preserves unknown header bytes from a known-good template and rewrites fields that are understood. A future standalone packer should derive every header field directly rather than requiring a template.

A new WindOI-generated ZNV is expected to be a useful next test because it should already contain valid internal ZNV records. Replacing `project.znv` in a known-good ZNX and rebuilding the outer record will test whether any remaining top-level header values are project-specific.

14 Source notes

Automation Organizer / WINDOI-NV4.exe: WNVManager.WriteDetailFileInfoForZNX;
CalculateChecksum;
CalculateChecksumFromArray;
CalculateChecksumBinary;
CopyFileDetailsInZNV.

Automation Organizer / WindLDR/Communications.dll: PDCommunication.MakeZNVFileForLinux;
PDCommunication.CalcSum4Bytes.

HMI runtime: HG2F.BIN,
idec_boot.BIN.

Whole-ZNX MD5 verification and os_update.znx.md5 handling.

Coreutils other Linux applications: file, grep, strings, dd, find, md5sum,
sed, ghex, xxd, readelf to name a few.